LLM Project to Build and Fine Tune a Large Language Model
Project Overview

Overview
In today's data-driven world, the ability to process and generate natural language text at scale has become a transformative force across industries. Large Language Models (LLMs) represent a cutting-edge advancement in natural language processing, enabling businesses to extract valuable insights, automate tasks, and enhance user experiences. By harnessing the power of LLMs, organizations can improve customer service, automate content creation, and gain a competitive edge in the digital landscape.

This project builds the foundation for Large Language Models by diving deep into the details of their inner workings. Moreover, It shows how to optimize their use through prompt engineering and fine-tuning techniques such as LoRA.

Prompt engineering techniques involve crafting specific instructions or queries given to the language model to influence its output will be introduced to guide LLMs in generating desired responses through zero-shot, one-shot, and few-shot inferences.

Fine-tuning entails training a pre-trained language model on a specific task or dataset to adapt it for a particular application. It explores full fine-tuning and Parameter Efficient Fine Tuning (PEFT), a technique that optimizes the fine-tuning process by focusing on a subset of the model's parameters, making it more resource-efficient.

The project also involves the application of Retrieval Augmented Generation (RAG) using OpenAI's GPT-3.5 Turbo, resulting in the development of a chatbot for online shopping for knowledge grounding. Knowledge grounding with Retrieval Augmented Generation (RAG) is implemented to mitigate hallucinations and provide trustworthy and reliable responses. This is achieved by incorporating information from external sources to validate and support the generated text.

For example, in the context of an e-commerce chatbot using RAG, knowledge grounding ensures that product information, availability, and prices are sourced from a trusted database or e-commerce platform. This prevents the chatbot from generating inaccurate or fictional details and instead provides responses based on real-world data.

Note: Please note that the use of the OpenAI API may require the utilization of allocated free credits or additional purchases (depending on the account status) for implementing the project. Kindly take note of the free credits limit provided for your usage.

Aim

The aim of this project is to provide a comprehensive understanding and practical experience in working with Large Language Models (LLMs). It covers fundamental concepts, advanced techniques, and practical applications of LLMs, to effectively leverage them for tasks such as text generation, fine-tuning, and building knowledge-grounded applications like a chatbot for online shopping.

Data Description

- The dialogue summarization exercises in this project utilize the knkarthick/dialogsum dataset from the Hugging Face library.
- For the purpose of demonstrating the development of a chatbot, data about apparel and paper products is provided. This dataset includes textual information about various apparel items and paper products.

Tech Stack

➔ Language: Python 3.8

➔ Libraries: transformers, datasets, torchdata, torch, streamlit, openai, langchain, unstructured, sentence-transformers, chromadb, evaluate, rouge_score, loralib, peft

Approach

- Introduction to LLMs and Basics:
    - Explanation of RNNs, transformers, attention mechanism, and Large Language Models (LLMs).
    - Illustration of use cases, including text generation, to establish foundational knowledge.
- Prompt Engineering:
    - Introduction to prompt engineering techniques, including zero, one, and few-shot inferences.
- Dialogue Summarization Task:
    - Implementation of prompt engineering on a dialogue summarization task, showcasing practical application.
- Fine Tuning and PEFT:
    - Implementing full fine-tuning and parameter-efficient fine-tuning (PEFT) on dialogue summarization.

    - Model evaluation using ROUGE metrics for performance assessment.
- RLHF (Reinforcement Learning from Human Feedback):
    - Explanation and exploration of the RLHF concept
- Retrieval Augmented Generation (RAG):
    - Practical implementation of RAG using OpenAI's GPT3.5 for creating a chatbot.
- Development of Chatbot Application:
    - Step-by-step guide on building a functional chatbot application for online shopping using RAG.

Cost Breakdown

Components

- Streamlit Chatbot with Opena AI and experimentation in OpenAI

Cost Breakdown as of January 17, 2025
The cost might have been updated, it is recommended to go through the following pricing pages to come up with updated guessestimates.
Open AI Pricing https://openai.com/api/pricing/

Component	Description	Usage	Estimated Cost
API Calls	OpenAI API (GPT-4 Turbo and GPT-3.5 Turbo) for RAG and chatbot responses	Models used: - GPT-3.5 Turbo (16K context): $0.0015 per 1K tokens (input), $0.002 per 1K tokens (output)- GPT-4 Turbo (8K context): $0.01 per 1K tokens (input), $0.03 per 1K tokens (output)	**Total Estimated Usage:**- 100 queries- Avg 1,000 tokens/query **Cost Estimate:**- GPT-3.5 Turbo: (input: $0.15, output: $0.20)- GPT-4 Turbo: (input: $1.00, output: $3.00)

Total Estimated Cost: ~$5 - $10 per project.

Recommendation: GPT-3.5 Turbo is more cost-effective for high-volume interactions, while GPT-4 Turbo provides superior responses but at a higher cost. Proper token usage monitoring is essential to optimize costs.

Experimentation should balance performance needs with budget constraints.For cost efficiency, consider using GPT-3.5 Turbo for routine chatbot interactions and GPT-4 Turbo for complex or critical tasks requiring higher accuracy.

If you're processing **one page of text (~1,000 tokens)**, the **total estimated cost would be around $0.05 - $0.50**, depending on the model used.

- **API Cost:**
  - **GPT-3.5 Turbo**: ~$0.003 per page ($0.0015 input + $0.002 output per 1,000 tokens)
  - **GPT-4 Turbo**: ~$0.04 per page ($0.01 input + $0.03 output per 1,000 tokens)
- **Cloud Costs (if applicable, for hosting and processing):** ~$0.01 - $0.05 per request
- **Storage Costs (saving results in a database/S3):** ~$0.001 per MB

Modular code overview:

```
├─ full
├─ images
├─ kb
│  ├─ apparel_products.txt
│  └─ paper_products.txt
├─ llm_app.py
├─ llm_labs.ipynb
├─ peft
├─ readme.md
└─ requirements.txt
```

Once you unzip the code.zip file, you can find the following files/folders.

1. full
2. images
3. kb
4. llm_app.py
5. llm_labs.ipynb
6. peft
7. readme.md
8. requirements.txt

1. The full folder contains files related to full fine-tuning of the language model.

2. The images folder stores images that are used for demonstration or visualization purposes within the project.

3. The kb (Knowledge Base) directory contains two text files, apparel_products.txt and paper_products.txt. These files serve as knowledge bases for the chatbot, providing information about apparel and paper products, respectively.

4. llm_app.py is the Streamlit application. In this application, users can interact with the chatbot. It is required to enter their OpenAI API key in this application.

5. The Jupyter notebook, named llm_labs.ipynb, serves as the central hub for all the explanatory material and codes related to different use cases. It contains sections for text generation, dialogue summarization, full fine-tuning, PEFT tuning, knowledge grounding etc.

6. The peft directory contains files related to the parameter efficient fine-tuning of the language model.

7. readme.md provides essential information about the project, including the Python version required to run the code and instructions for execution.

8. requirements.txt contains a list of all the required libraries and their versions needed to run the project.

Project Takeaways

1. What are Large Language Models (LLMs)?
2. Understand RNNs, their application, and limitations
3. What are Transformers?
4. Understand Attention Mechanism and Transformer Architecture
5. What are tokenizers?
6. What are embeddings?
7. Generating text and summarizing dialogues using LLMs
8. What is Prompt Engineering?

9. Learn optimization techniques like Prompt Engineering, Fine-Tuning, and PEFT
10. Fine-tune LLMs for improved performance on tasks
11. Evaluate model performance using the ROUGE metric
12. Understand RLHF for improved model output.
13. What is knowledge grounding?
14. Implement Retrieval Augmented Generation (RAG) for knowledge grounding
15. Building a chatbot application for an e-commerce use case